

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВОРОНЕЖСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «ВГУ»)

Факультет компьютерных наук
Кафедра программирования и информационных технологий

«Каталог квартир» — мобильное приложение для риелторов

Курсовая работа по дисциплине
«Учебная практика (научно-исследовательская работа)»
09.03.02 Информационные системы и технологии
Программная инженерия в информационных системах

6 семестр 2025/2026 учебного года
09.03.02 Информационные системы и технологии

Зав. кафедрой	_____	д. ф.-м. н., доцент Махортов С. Д.
Обучающийся	_____	ст. 3 курса Дворцов С. Д.
Руководитель	_____	ассистент Ушаков В. А.

Воронеж 2026

СОДЕРЖАНИЕ

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ.....	4
ВВЕДЕНИЕ.....	6
1 Теоретическая часть.....	8
1.1 Анализ предметной области и актуальность.....	8
1.2 Обзор аналогов	8
1.2.1 ЦИАН	8
1.2.2 AmoCRM	9
1.2.3 Сравнительный анализ	9
1.3 Функциональные и нефункциональные требования	10
2 Проектирование.....	12
2.1 Выбор технологического стека.....	12
2.2 Архитектура решения	12
2.3 Проектирование базы данных.....	13
2.4 Проектирование REST API	14
3 Реализация.....	16
3.1 Реализация серверной части	16
3.2 Реализация Android-клиента	17
3.2.1 Аутентификация и обновление токенов	17
3.2.2 Список квартир и пагинация.....	18
3.2.3 Аналитика и графики	19
3.3 Дизайн приложения	20
3.3.1 Список квартир.....	20
3.3.2 Фильтрация объектов.....	21
3.3.3 Детальный просмотр объекта	21
3.3.4 Добавление квартиры	22
3.3.5 Аналитика	23
3.3.6 Ипотечный калькулятор	24
3.3.7 Список избранного.....	25
3.3.8 Профиль пользователя.....	26
3.4 Тестирование	27
3.5 Развёртывание и конфигурация.....	29
ЗАКЛЮЧЕНИЕ	32

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	33
--	----

ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

В настоящей курсовой работе применяются следующие термины с соответствующими определениями:

Android — мобильная операционная система на базе ядра Linux, разрабатываемая компанией Google. Является целевой платформой разрабатываемого приложения.

API (Application Programming Interface) — программный интерфейс взаимодействия между приложениями. В данной работе реализован в форме REST API.

Clean Architecture — архитектурный подход, предполагающий разделение кода на независимые слои данных (data) и представления (ui) с однонаправленным потоком зависимостей.

CRM (Customer Relationship Management) — система управления взаимоотношениями с клиентами. В контексте данной работы — инструмент учёта объектов недвижимости и статусов сделок.

CRUD (Create, Read, Update, Delete) — базовые операции над данными: создание, чтение, обновление и удаление.

JWT (JSON Web Token) — открытый стандарт для безопасной передачи информации в виде JSON-объекта. Используется для аутентификации пользователей.

MVVM (Model–View–ViewModel) — архитектурный паттерн, разделяющий UI и бизнес-логику посредством ViewModel, обеспечивающий тестируемость и реактивность.

ORM (Object-Relational Mapping) — технология преобразования объектов приложения в записи реляционной базы данных. В работе используется Exposed ORM.

Paging 3 — библиотека Android Jetpack для постраничной загрузки данных. Обеспечивает бесконечную прокрутку без полной загрузки данных в память.

REST (Representational State Transfer) — стиль архитектуры, основанный на протоколе HTTP, с использованием методов GET, POST, PUT, DELETE.

UDF (Unidirectional Data Flow) — принцип однонаправленного потока данных: View генерирует события, ViewModel публикует новое состояние через StateFlow.

ВВЕДЕНИЕ

Современный рынок недвижимости характеризуется высокой динамичностью и значительным объёмом информации, которую специалисты — риелторы и агентства — вынуждены обрабатывать ежедневно. По данным отраслевых аналитиков, на крупнейших российских платформах публикуются сотни тысяч объявлений в месяц. Вместе с тем инструментарий для оперативного учёта объектов остаётся недостаточно развитым: большинство агентов ведут базу в электронных таблицах или устаревших CRM-системах, не оптимизированных для мобильной работы.

Мобильные приложения позволяют обеспечить риелторов актуальной информацией в режиме реального времени на месте показа объекта. Однако готовые решения либо являются дорогостоящими (AmoCRM, Битрикс24), либо не предоставляют необходимого сочетания функций: учёта объектов, управления статусами сделок и встроенной аналитики.

Цель работы — разработать клиент-серверное мобильное приложение «Каталог квартир» для риелторов, обеспечивающее учёт объектов недвижимости, управление статусами сделок и анализ продаж.

Задачи работы:

- изучить предметную область мобильных CRM-инструментов и провести анализ аналогов;
- спроектировать клиент-серверную архитектуру (Android + Ktor + PostgreSQL);
- реализовать JWT-аутентификацию с автоматическим обновлением токенов;
- разработать модули: список с пагинацией, детали, создание и редактирование объектов;
- реализовать аналитический экран с диаграммами и ипотечный калькулятор;
- провести функциональное тестирование и зафиксировать результаты.

Объект исследования — процесс управления информацией о недвижимости в агентствах и у частных риелторов. Предмет исследования — методы и технологии разработки мобильных клиент-серверных приложений для Android.

Практическая значимость: разработан функционально полный инструмент, готовый к развёртыванию в реальном агентстве недвижимости. Сборка проекта завершается без ошибок (BUILD SUCCESSFUL), APK готов к установке на Android 8.0 и выше.

1 Теоретическая часть

1.1 Анализ предметной области и актуальность

Предметной областью работы является автоматизация деятельности риелторов с помощью мобильного ПО. Ключевые бизнес-процессы, подлежащие автоматизации:

- ведение каталога объектов с полными характеристиками (адрес, площадь, этаж, цена, тип);
- управление жизненным циклом сделки через изменение статусов: Доступна → Бронь → Продана или Отменена;
- регистрация истории изменений статусов с указанием агента и комментария;
- анализ портфеля: распределение по статусам, динамика сделок, цены по районам;
- ипотечный калькулятор для оценки финансовых параметров объекта;
- персональный список избранных объектов.

Актуальность разработки обусловлена ростом объёма данных, обрабатываемых агентами, недоступностью или дороговизной существующих специализированных решений для небольших агентств, а также развитием экосистемы Android Jetpack, делающей разработку производительных мобильных инструментов значительно проще.

1.2 Обзор аналогов

В ходе анализа предметной области рассмотрены два программных продукта, наиболее близких по функциональности к разрабатываемому приложению.

1.2.1 ЦИАН

ЦИАН — крупнейшая российская платформа для объявлений о недвижимости. Мобильное приложение поддерживает полнотекстовый поиск,

фильтры, отображение на карте и ипотечный калькулятор. Ориентировано на покупателя, а не на агента: отсутствует управление статусами собственных сделок, история изменений и аналитика портфеля. Расширенная аналитика доступна только на платных корпоративных тарифах.

1.2.2 AmoCRM

AmoCRM — универсальная CRM-система, применяемая в агентствах недвижимости. Поддерживает воронки сделок, управление контактами и аналитику. Основные недостатки: стоимость от 599 Р/мес. на пользователя, избыточный функционал для небольших агентств, отсутствие ипотечного калькулятора и специализированных функций работы с объектами.

1.2.3 Сравнительный анализ

На основании анализа составлена таблица сравнения аналогов (таблица 1).

Таблица 1 — Сравнительный анализ аналогов

Параметр	ЦИАН	AmoCRM	«Каталог квартир»
Мобильный клиент Android	✓	✓	✓
Управление статусами сделок	—	✓	✓
Аналитика / графики	частично	✓	✓
Ипотечный калькулятор	✓	—	✓
История изменений статусов	—	частично	✓

Избранные объекты	✓	–	✓
REST API + JWT	–	–	✓
Стоимость	бесплатно	от 599 Р/мес.	открытый код

Ни один из аналогов не сочетает специализированную работу с объектами недвижимости, встроенную аналитику, ипотечный калькулятор и открытый исходный код. Разрабатываемое приложение устраняет данный пробел.

1.3 Функциональные и нефункциональные требования

На основе анализа предметной области и аналогов сформулированы требования к приложению.

Функциональные требования:

- регистрация и авторизация с JWT-аутентификацией;
- просмотр списка квартир с поиском по адресу в реальном времени;
- фильтрация по статусу, типу объявления, числу комнат; сортировка по цене, площади, дате;
- просмотр детальной информации: адрес, площадь, фото, цена за м², просмотры, дата;
- создание и редактирование объявления с валидацией полей;
- изменение статуса сделки с комментарием и записью истории;
- добавление в избранное и просмотр списка избранных;
- аналитика: кольцевая диаграмма по статусам, динамика сделок, средние цены по районам;
- ипотечный калькулятор с аннуитетным расчётом.

Нефункциональные требования:

- время отклика API не более 2 секунд при нагрузке до 50 пользователей;

- хранение токенов в зашифрованном виде (DataStore Preferences);
- прозрачное автоматическое обновление access-токена (TokenAuthenticator);
- поддержка Android API 26 (Android 8.0) и выше;
- соответствие UI принципам Material Design 3.

2 Проектирование

2.1 Выбор технологического стека

При выборе технологий руководствовались критериями: статическая типизация, актуальность, зрелая экосистема библиотек, поддержка корутин для асинхронности и возможность единого языка для клиента и сервера.

Язык Kotlin выбран для обеих частей системы. Это официально рекомендуемый Google язык для Android (с 2019 года), обладающий null-safety, корутинами и лаконичным синтаксисом. Единый язык упрощает поддержку кодовой базы.

Стек Android-клиента:

- Clean Architecture + MVVM — разделение на слои data/ и ui/;
- Retrofit 2.11 + OkHttp 4.12 + kotlinx-serialization — сетевой слой;
- Paging 3 — бесконечная подгрузка списков;
- DataStore Preferences — хранение токенов и избранного;
- Coil 2.7.0 — загрузка изображений;
- MPAndroidChart 3.1.0 — диаграммы;
- Navigation Component + BottomNavigationView — навигация;
- Kotlin Coroutines + StateFlow / SharedFlow — асинхронность.

Стек серверной части:

- Ktor 2.3.12 (Netty) — асинхронный веб-фреймворк;
- Exposed ORM — типобезопасный доступ к PostgreSQL;
- PostgreSQL + HikariCP — СУБД и пул соединений;
- JWT (com.auth0:java-jwt) + bcrypt — аутентификация;
- kotlinx.serialization + Content Negotiation — сериализация.

2.2 Архитектура решения

Система построена по трёхзвенной клиент-серверной модели: Android-клиент взаимодействует с Ktor-сервером по REST API (HTTP/JSON), сервер

обращается к PostgreSQL через Exposed ORM. Такая архитектура обеспечивает независимость клиента и сервера.

Слой данных Android-клиента (data/) включает: api/ (ApiService, AuthInterceptor, TokenAuthenticator), local/ (TokenManager, FavoritesManager), models/ (DTO), paging/ (PagingSource), repository/ (репозитории). Слой представления (ui/) содержит фрагменты и ViewModel для модулей: auth/, apartments/, analytics/, favorites/, mortgage/, profile/.

ViewModel реализует паттерн UDF: View отправляет события → ViewModel обрабатывает через репозитории → публикует состояние через StateFlow<UiState>Loading|Success|Error>. Разовые события (Snackbar, переход) — через SharedFlow.

Серверная часть: routes/ → service/ → repository/ → Exposed ORM. Слоистая организация обеспечивает высокую связность внутри слоя и низкую — между слоями.

2.3 Проектирование базы данных

База данных PostgreSQL (схема android) содержит 6 таблиц (таблица 2). Типы listing_type {sale, rent} и status {available, reserved, sold, cancelled} реализованы как PostgreSQL ENUM, что исключает некорректные значения на уровне СУБД.

Таблица 2 — Таблицы базы данных

Таблица	Назначение	Ключевые поля
users	Учётные записи агентов	id, full_name, email, phone, password_hash
refresh_tokens	Сессии пользователей	id, user_id → users, token, expires_at, revoked

buildings	Здания	id, city, district, street, house_number, total_floors
apartments	Объявления	id, building_id, agent_id, floor, rooms, total_area, price, listing_type*, status*
apartment_photos	Фотографии	id, apartment_id → apartments, url, is_main, sort_order
apartment_status_history	История статусов	id, apartment_id, changed_by, old_status, new_status, comment

2.4 Проектирование REST API

REST API принимает запросы на порту 8080. Все защищённые эндпоинты требуют заголовка Authorization: Bearer <access_token> (таблица 3).

Таблица 3 — REST API

Метод	Эндпоинт	Описание
POST	/auth/register	Регистрация пользователя
POST	/auth/login	Вход; возвращает access + refresh токены
POST	/auth/refresh	Обновление access-токена
POST	/auth/logout	Выход, отзыв refresh-токена
GET	/apartments	Список с фильтрацией и пагинацией
GET	/apartments/{id}	Детали объекта

POST	/apartments	Создание объявления
PUT	/apartments/{id}	Редактирование / смена статуса
DELETE	/apartments/{id}	Удаление объекта
GET	/analytics/stats	Агрегированная статистика

Параметры фильтрации GET /apartments: status, listingType, rooms, minPrice, maxPrice, search, sortBy, sortOrder, page, pageSize. Ответ содержит поля data, total, page, totalPages.

3 Реализация

3.1 Реализация серверной части

Серверная часть реализована на Ktor 2.3.12 с движком Netty. Точка входа — функция `main()`, регистрирующая плагины и запускающая сервер на порту 8080. Листинг 1 демонстрирует конфигурацию маршрутизации квартир.

Листинг 1 — Маршруты эндпоинта /apartments (Kotlin, Ktor)

```
fun Application.configureApartmentRoutes(service:
ApartmentService) {
    routing {
        authenticate("jwt") {
            route("/apartments") {
                get {
                    val page =
                        call.request.queryParameters["page"]?.toIntOrNull()
                        ?: 1
                    val size =
                        call.request.queryParameters["pageSize"]?.toIntOrNull()
                        ?: 20
                    val status = call.request.queryParameters["status"]
                    val result = service.getApartments(page, size,
                        status)
                    call.respond(result)
                }
                post {
                    val dto = call.receive<ApartmentCreateDto>()
                    val agentId =
                        call.principal<JWTPrincipal>()!!.subject!!
                    val created = service.createApartment(dto, agentId)
                    call.respond(HttpStatusCode.Created, created)
                }
            }
        }
    }
}
```

Аутентификация строится на паре JWT-токенов. При входе сервер выдаёт access-токен (TTL 15 мин) и refresh-токен (TTL 30 дней). Листинг 2 показывает генерацию токена.

Листинг 2 — Генерация JWT access-токена (Kotlin)

```
fun generateAccessToken(userId: String): String {
    return JWT.create()
        .withSubject(userId)
        .withIssuer(jwtConfig.issuer)
        .withAudience(jwtConfig.audience)
        .withExpiresAt(Date(System.currentTimeMillis() + 15 * 60 * 1000))
        .sign(Algorithm.HMAC256(jwtConfig.secret))
}
```

3.2 Реализация Android-клиента

3.2.1 Аутентификация и обновление токенов

Сетевой слой организован через два перехватчика OkHttp. AuthInterceptor добавляет JWT-заголовок к каждому запросу. TokenAuthenticator обрабатывает ответы 401: обновляет токен и повторяет запрос. Листинг 3 показывает реализацию TokenAuthenticator.

Листинг 3 — TokenAuthenticator (Kotlin, OkHttp)

```
class TokenAuthenticator(
    private val tokenManager: TokenManager,
    private val authApi: AuthApi
) : Authenticator {

    override fun authenticate(route: Route?, response: Response): Request? {
        if (responseCount(response) >= 2) return null // защита от цикла
        val refreshToken = runBlocking {
            tokenManager.getRefreshToken()
        }
        ?: return null
        val newTokens = runBlocking {
            authApi.refresh(RefreshDto(refreshToken))
        }
        runBlocking { tokenManager.saveTokens(newTokens) }
        return response.request.newBuilder()
            .header("Authorization", "Bearer ${newTokens.accessToken}")
            .build()
    }
}
```

```
}
```

3.2.2 Список квартир и пагинация

Список реализован через Paging 3. Листинг 4 демонстрирует PagingSource с поддержкой фильтров.

Листинг 4 — ApartmentsPagingSource (Kotlin, Paging 3)

```
class ApartmentsPagingSource(
    private val api: ApiService,
    private val filters: ApartmentFilters
) : PagingSource<Int, ApartmentDto>() {

    override suspend fun load(params: LoadParams<Int>):
    LoadResult<Int, ApartmentDto> {
        val page = params.key ?: 1
        return try {
            val response = api.getApartments(
                page = page, pageSize = params.loadSize,
                status = filters.status, search = filters.query
            )
            LoadResult.Page(
                data = response.data,
                prevKey = if (page == 1) null else page - 1,
                nextKey = if (page >= response.totalPages) null else
                    page + 1
            )
        } catch (e: Exception) {
            LoadResult.Error(e)
        }
    }
}
```

В ViewModel поиск реализован через flatMapLatest, что обеспечивает автоматическую отмену предыдущего запроса при изменении строки поиска:

Листинг 5 — Поиск в реальном времени (Kotlin, ViewModel)

```
val apartments = searchQuery
    .debounce(300)
    .flatMapLatest { query ->
```

```

Pager(PagingConfig(pageSize = 20)) {
    ApartmentsPagingSource(api, filters.copy(query = query))
}.flow
}
.cachedIn(viewModelScope)

```

3.2.3 Аналитика и графики

Аналитический экран загружает данные через GET /analytics/stats и отображает три вида диаграмм средствами MPAndroidChart 3.1.0. Листинг 6 демонстрирует настройку кольцевой диаграммы.

Помимо диаграмм, в верхней части экрана отображаются два числовых счётчика: общее число квартир в системе и суммарное количество просмотров. При загрузке данных показывается CircularProgressIndicator; при ошибке сети — Snackbar с кнопкой «Повторить». Данные кешируются в ViewModel, поэтому при переключении между вкладками повторный сетевой запрос не выполняется.

Столбчатая диаграмма (BarChart) отображает количество завершённых сделок за каждый из последних 12 месяцев. По оси X — сокращённые названия месяцев, по оси Y — число сделок. Горизонтальная гистограмма (HorizontalBarChart) показывает средние цены объектов по районам, отсортированные по убыванию; выводятся топ-8 районов с наибольшим количеством объявлений.

Листинг 6 — Настройка PieChart (Kotlin, MPAndroidChart)

```

private fun setupPieChart(stats: StatusStats) {
    val entries = listOf(
        PieEntry(stats.available.toFloat(), "Доступна"),
        PieEntry(stats.reserved.toFloat(), "Бронь"),
        PieEntry(stats.sold.toFloat(), "Продана")
    )
    val dataSet = PieDataSet(entries, "").apply {
        colors = listOf(Color.GREEN, Color.parseColor("#FF9800"), Color.GRAY)
        valueTextSize = 12f
    }
}

```

```

    }
    binding.pieChart.apply {
        data = PieData(dataSet)
        isDrawHoleEnabled = true
        holeRadius = 40f
        invalidate()
    }
}

```

3.3 Дизайн приложения

3.3.1 Список квартир

Главный экран приложения отображает список объектов с цветовой индикацией статуса и типа объявления. В верхней части — строка поиска по адресу. Иконки в Toolbar открывают список избранного и панель фильтрации.

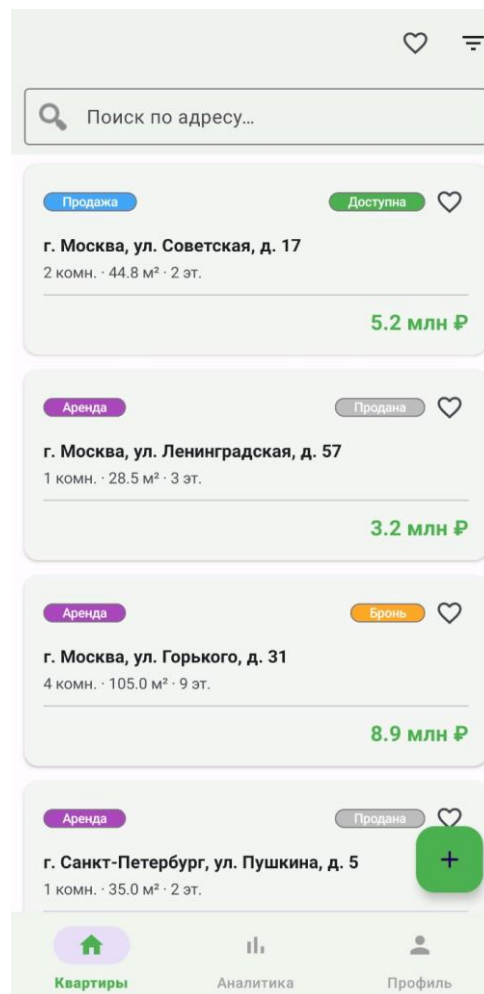


Рисунок 1 — Список квартир

3.3.2 Фильтрация объектов

Панель фильтрации реализована как BottomSheetDialogFragment и позволяет выбрать статус объекта, тип объявления, количество комнат и порядок сортировки. Кнопка «Применить» перезагружает список с новыми параметрами. Кнопка «Сбросить» возвращает фильтры к значениям по умолчанию.

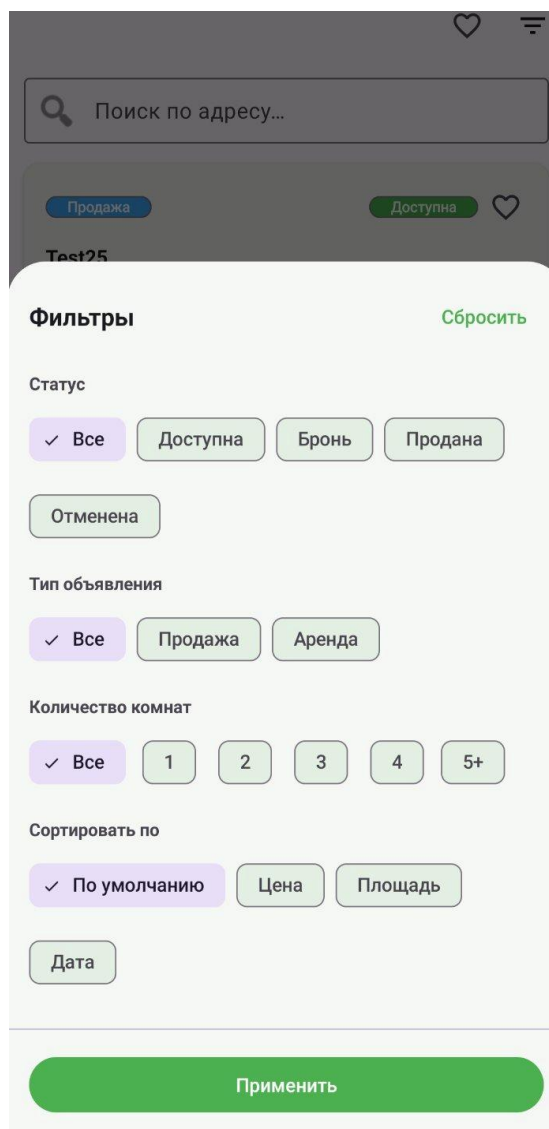


Рисунок 2 — Панель фильтрации

3.3.3 Детальный просмотр объекта

Экран детального просмотра отображает полную информацию: адрес, число комнат, площадь, этаж, цену и цену за м², счётчик просмотров и дату

добавления. Кнопка «Изменить» открывает диалог смены статуса с обязательным комментарием. Кнопка «Ипотека» открывает калькулятор с предзаполненной ценой.

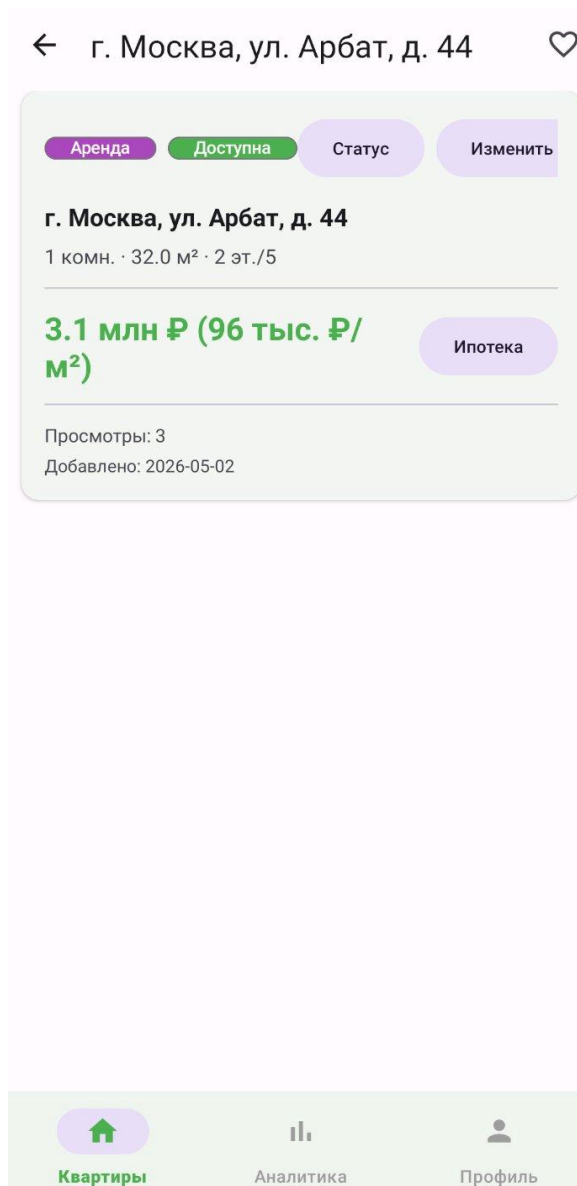


Рисунок 3 — Детальный просмотр объекта

3.3.4 Добавление квартиры

Форма добавления содержит поля: полный адрес, этаж, общее число этажей, количество комнат, площадь, цена, переключатель «Продажа / Аренда» и необязательное описание. Перед сохранением выполняется

клиентская валидация. Та же форма используется для редактирования существующего объекта.

← Новая квартира

Полный адрес

Этаж

Всего этажей

Комнат

Площадь, м²

Цена, ₽

Тип объявления

Продажа

Аренда

Описание (необязательно)

Сохранить

Квартиры

Аналитика

Профиль

Рисунок 4 — Форма добавления квартиры

3.3.5 Аналитика

Экран аналитики отображает сводную статистику портфеля. В верхней части — счётчики (28 квартир, 54 просмотра). Ниже — кольцевая диаграмма по статусам и столбчатая диаграмма динамики сделок по месяцам.



Рисунок 5 — Экран аналитики

3.3.6 Ипотечный калькулятор

Калькулятор позволяет оценить стоимость кредита прямо при показе объекта. Пользователь вводит цену, первоначальный взнос, ставку и срок — результат пересчитывается в реальном времени. На скриншоте: цена 3 100 000 Р, взнос 620 000 Р (20%), ставка 16%, срок 20 лет → платёж 34 тыс. Р/мес., переплата 5,8 млн Р.

The screenshot shows a mobile application interface for a mortgage calculator. At the top, there is a back arrow. Below it, a light green rounded rectangle contains three input fields: 'Цена, ₽' with the value '3100000', 'Первоначальный взнос, ₽' with the value '620000', and two side-by-side fields for 'Ставка, % годовых' (16) and 'Срок кредита, лет' (20). Below this, another light green rounded rectangle displays the 'Ежемесячный платёж' as '34 тыс. ₽' in green. Underneath, a table shows the total costs: 'Сумма кредита' (2.5 млн ₽), 'Переплата по кредиту' (5.8 млн ₽), and 'Итого выплат банку' (8.3 млн ₽). At the bottom, a navigation bar with a light green background contains three icons: a house for 'Квартиры', a bar chart for 'Аналитика', and a person for 'Профиль'.

Ежемесячный платёж	
34 тыс. ₽	
Сумма кредита	2.5 млн ₽
Переплата по кредиту	5.8 млн ₽
Итого выплат банку	8.3 млн ₽

Квартиры	Аналитика	Профиль
----------	-----------	---------

Рисунок 6 — Ипотечный калькулятор

3.3.7 Список избранного

Экран избранного отображает объекты, добавленные агентом в персональный список. Список хранится в DataStore Preferences и доступен без сетевого подключения. Добавление и удаление из избранного происходят мгновенно при нажатии иконки сердца.

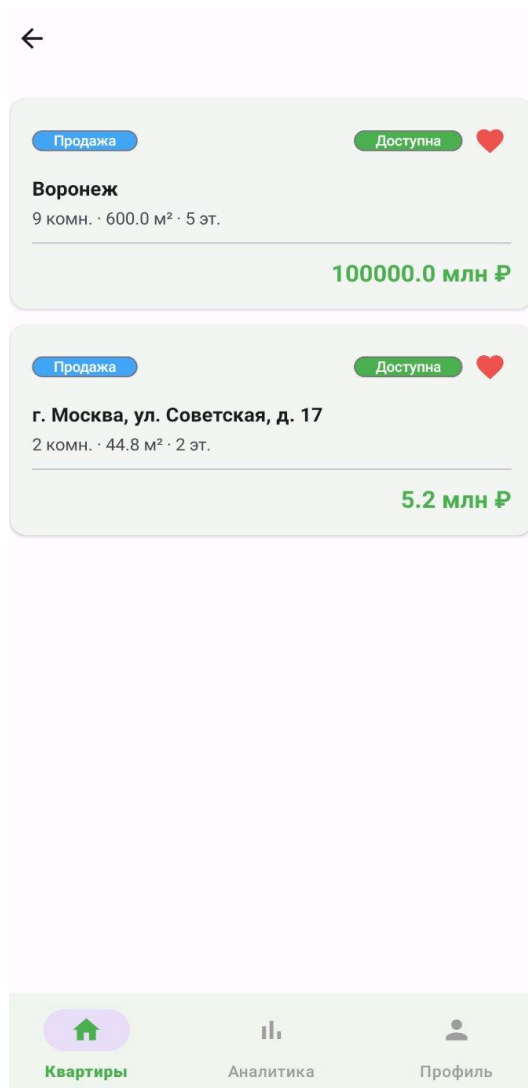


Рисунок 7 — Список избранного

3.3.8 Профиль пользователя

Экран профиля отображает имя пользователя и адрес электронной почты. Кнопка «Выйти из аккаунта» вызывает POST /auth/logout (отзыв refresh-токена), очищает DataStore и перенаправляет на экран входа.

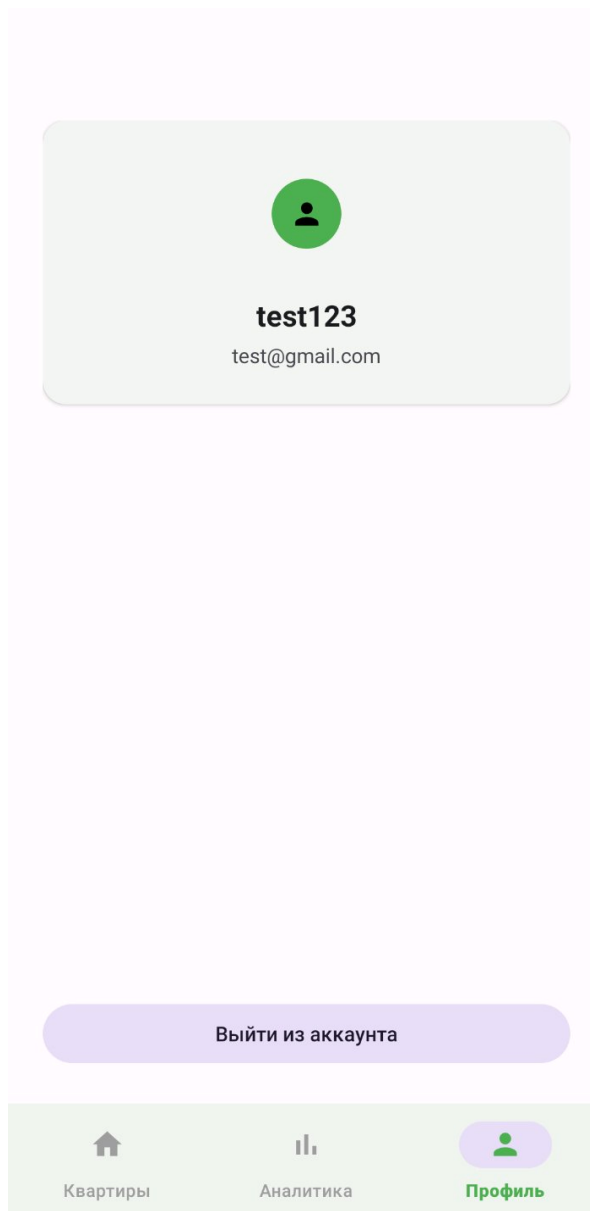


Рисунок 8 — Профиль пользователя

3.4 Тестирование

Тестирование проводилось в форме ручного функционального тестирования на физическом устройстве под управлением Android 13 (API 33) и в эмуляторе Android Studio (API 30). Каждый сценарий выполнялся не менее трёх раз для исключения случайных отказов. Таблица 4 содержит полный перечень протестированных сценариев и результаты.

Помимо функционального тестирования, был проведён ряд дополнительных проверок. Во-первых, проверена корректность

автоматического обновления access-токена: при принудительном истечении токена на сервере клиент прозрачно получал новый и повторял запрос без участия пользователя. Во-вторых, проверена устойчивость к ошибкам сети: при отключении соединения PagingSource корректно возвращал LoadResult.Error, LoadStateAdapter отображал кнопку «Повторить». В-третьих, проверена корректность работы при пересоздании активности (поворот экрана): данные сохранялись в ViewModel и не запрашивались повторно из сети.

Таблица 4 — Результаты функционального тестирования

Сценарий	Ожидаемый результат	Результат
Регистрация с корректными данными	Создание аккаунта, переход на список	Соответствует
Вход с неверным паролем	Сообщение об ошибке	Соответствует
Поиск по адресу	Фильтрация в реальном времени	Соответствует
Фильтр по статусу	Только объекты с выбранным статусом	Соответствует
Сортировка по цене	Список упорядочен по возрастанию	Соответствует
Создание объявления	Квартира появляется в списке	Соответствует
Редактирование объявления	Изменения сохранены и отображены	Соответствует

Смена статуса с комментарием	Статус обновлён, история записана	Соответствует
Добавление в избранное	Объект доступен в избранном	Соответствует
Расчёт ипотеки	Корректный расчёт платежа	Соответствует
Истечение access-токена	Прозрачное обновление, запрос повторён	Соответствует
Сборка проекта (gradle)	BUILD SUCCESSFUL, APK готов	Соответствует

Все 12 протестированных сценариев завершились с ожидаемым результатом. Сборка проекта без ошибок (BUILD SUCCESSFUL). APK готов к установке на устройства Android 8.0+.

3.5 Развёртывание и конфигурация

Серверная часть может быть развёрнута на любом хосте с поддержкой JVM. Для запуска необходимы: JDK 17+, PostgreSQL 14+ и переменные окружения с параметрами подключения к БД и секретом JWT. Листинг 7 демонстрирует конфигурационный файл Ktor.

Листинг 7 — Конфигурация Ktor (application.conf, HOCON)

```
ktor {
  deployment {
    port = 8080
    port = ${?PORT}
  }
  application {
    modules = [ com.example.ApplicationKt.module ]
  }
}
```

```

    }
}
database {
    url      = ${DB_URL}
    user     = ${DB_USER}
    password = ${DB_PASSWORD}
}
jwt {
    secret    = ${JWT_SECRET}
    issuer    = "realty-app"
    audience  = "realty-users"
}

```

Схема базы данных создаётся автоматически при первом запуске сервера через вызов `SchemaUtils.createMissingTablesAndColumns()` для всех таблиц `Exposed`. PostgreSQL ENUM-типы создаются отдельной миграцией SQL-скриптом.

Для Android-клиента базовый URL сервера вынесен в константу в модуле `NetworkModule`. В production-окружении предполагается использование HTTPS и проверка SSL-сертификата. На этапе разработки использовалась связка Android-эмулятора с хост-машиной через адрес `10.0.2.2:8080`.

Листинг 8 демонстрирует инициализацию `Retrofit` в модуле `NetworkModule`.

Листинг 8 — Конфигурация Retrofit (Kotlin)

```

object NetworkModule {

    private const val BASE_URL = "http://10.0.2.2:8080/"

    fun provideRetrofit(tokenManager: TokenManager): ApiService {
        val client = OkHttpClient.Builder()
            .addInterceptor(AuthInterceptor(tokenManager))
            .authenticator(TokenAuthenticator(tokenManager,
                provideAuthApi()))
            .connectTimeout(30, TimeUnit.SECONDS)
            .readTimeout(30, TimeUnit.SECONDS)
            .build()
        return Retrofit.Builder()
    }
}

```

```
        .baseUrl(BASE_URL)
        .client(client)
        .addConverterFactory(
            Json { ignoreUnknownKeys = true }.asConverterFactory(
                "application/json".toMediaType()
            )
        ).build()
        .create(ApiService::class.java)
    }
}
```

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы разработано функционально полное клиент-серверное мобильное приложение «Каталог квартир» для автоматизации деятельности риелторов.

Решены все поставленные задачи:

- проведён анализ предметной области и обзор аналогов (ЦИАН, AmoCRM); выявлены функциональные ограничения каждого;
- спроектирована трёхзвенная архитектура: Android + Ktor 2.3 + PostgreSQL; разработаны схема БД (6 таблиц) и REST API (10 эндпоинтов);
- реализована JWT-аутентификация с прозрачным обновлением токенов (TokenAuthenticator);
- разработаны модули: список с пагинацией Paging 3 и поиском, детали, создание, редактирование, история статусов;
- реализованы аналитический экран (MPAndroidChart) и ипотечный калькулятор;
- проведено функциональное тестирование 12 сценариев — замечаний не выявлено.

Объём кода: ~2 800 строк Kotlin (Android) + ~1 300 строк (backend).

Приложение поддерживает Android 8.0+.

Перспективы развития: офлайн-режим (Room + RemoteMediator), push-уведомления (Firebase), карта объектов (Яндекс.Карты / OSM), CRM-функционал (клиенты, звонки, показы), тёмная тема и адаптивный дизайн для планшетов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Android Developers. Paging 3 library overview [Электронный ресурс]. — Режим доступа: <https://developer.android.com/topic/libraries/architecture/paging/v3-overview> (дата обращения: 01.05.2026).
2. Android Developers. DataStore [Электронный ресурс]. — Режим доступа: <https://developer.android.com/topic/libraries/architecture/datastore> (дата обращения: 01.05.2026).
3. JetBrains. Ktor Documentation [Электронный ресурс]. — Режим доступа: <https://ktor.io/docs> (дата обращения: 15.04.2026).
4. JetBrains. Exposed Framework [Электронный ресурс]. — Режим доступа: <https://github.com/JetBrains/Exposed> (дата обращения: 15.04.2026).
5. PhilJay. MPAndroidChart [Электронный ресурс]. — Режим доступа: <https://github.com/PhilJay/MPAndroidChart> (дата обращения: 20.04.2026).
6. Coil contributors. Coil: Coroutine Image Loader [Электронный ресурс]. — Режим доступа: <https://coil-kt.github.io/coil> (дата обращения: 20.04.2026).
7. RFC 7519. JSON Web Token (JWT) / M. Jones, J. Bradley, N. Sakimura. — Internet Engineering Task Force, 2015.
8. PostgreSQL Documentation. PostgreSQL 15 [Электронный ресурс]. — Режим доступа: <https://www.postgresql.org/docs/15> (дата обращения: 10.04.2026).
9. Martin R. C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. — Prentice Hall, 2017. — 432 с.
10. Google. Material Design 3 [Электронный ресурс]. — Режим доступа: <https://m3.material.io> (дата обращения: 01.05.2026).
11. Square Inc. Retrofit [Электронный ресурс]. — Режим доступа: <https://square.github.io/retrofit> (дата обращения: 01.05.2026).

12. Android Developers. Navigation Component [Электронный ресурс]. — Режим доступа: <https://developer.android.com/guide/navigation> (дата обращения: 01.05.2026).